

Manuale amministratore

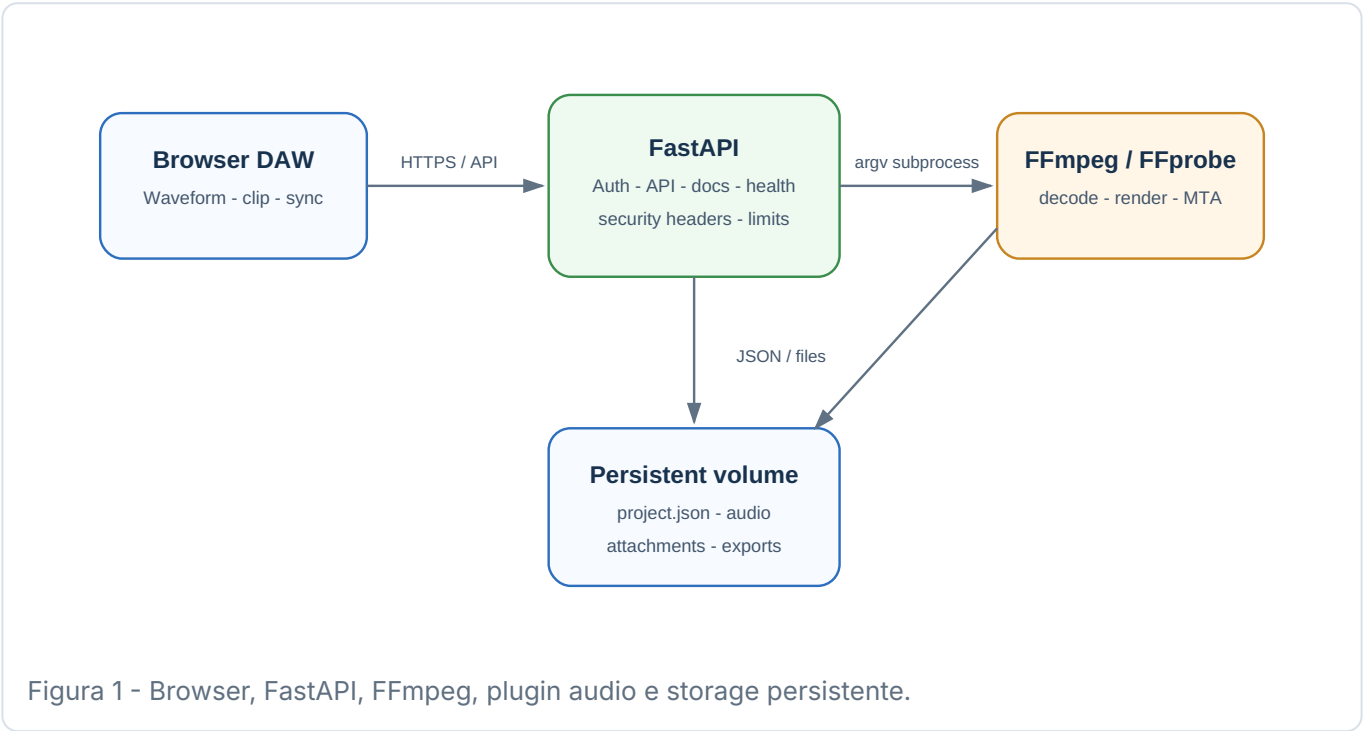
Versione 0.2.0-60 · build 20261001-18:42:54

Repository: <https://github.com/desalvo/mta-audio-editor> ·
EUPL-1.2

Indice

- 1. Architettura
- 2. Docker
- 3. Kubernetes
- 4. Configurazione
- 5. Demucs/stem plugin
- 6. Pipeline audio
- 7. Sicurezza
- 8. CI/CD
- 9. Operazioni
- 10. Backup
- 11. Release
- 12. Responsabilità production
- 13. Plugin custom e Auto Mix
- Appendice A. Formato proprietario MTA
- Appendice B. Progetto e architettura

1. Architettura



Componente	Responsabilità
FastAPI/Uvicorn	UI, API, autenticazione, health e documentazione.
FFmpeg/FFprobe	Probe, rendering clip, insert, summing, export e container MTA.
Demucs plugin	Separazione MP3 in stems indipendenti.
/data/projects	Project JSON, sorgenti, attachment, model cache ed export.

2. Docker

```
export MTA_ADMIN_PASSWORD='password-lunga-casuale'
export MTA_ADMIN_EMAIL='admin@example.com'
docker compose up -d --build
```

L'immagine production include FFmpeg e, per default, il plugin Demucs. Il container gira come UID/GID 10001, senza capability e con no-new-privileges. La root filesystem è read-only nel deployment consigliato.

Per una build ridotta senza stem splitter:

```
docker build --build-arg INSTALL_STEMS=false -t mta-audio-editor:core
```

3. Kubernetes / Kustomize

Obbligatorio: proteggere il Secret. `secret.example.yaml` non è incluso nel Kustomization di base; in produzione preferire External Secrets, Sealed Secrets, Vault o equivalente.

Le risorse sono separate in Namespace, PVC, Deployment e Service sotto `k8s/base`. Gli esempi Ingress sono disponibili in `k8s/overlays/nginx` e `k8s/overlays/haproxy`.

```
kubectl apply -k k8s/overlays/nginx  
# oppure  
kubectl apply -k k8s/overlays/haproxy
```

Il wizard standalone `scripts/k8s-wizard.py` genera manifest locali completi chiedendo username/password amministrativi, StorageClass, namespace e nodeSelector. Ricorda gli ultimi valori in un file locale `0600`, può produrre un Ingress NGINX/HAProxy e controlla automaticamente una versione più recente su GitHub; se si aggiorna sostituisce atomicamente il proprio file e si riavvia da solo.

```
curl -fsSL mta-k8s-wizard.py https://raw.githubusercontent.com/desal  
python3 mta-k8s-wizard.py
```

Il PVC montato su `/data/projects` contiene anche la cache Demucs/Torch, così i modelli non devono essere riscaricati a ogni restart.

4. Configurazione

Variabile	Default	Descrizione
MTA_ADMIN_USERNAME	admin	Username bootstrap del primo amministratore.
MTA_ADMIN_PASSWORD	nessuno	Password bootstrap del primo amministratore.
MTA_ADMIN_EMAIL	nessuno	Email obbligatoria per creare il primo amministratore.
MTA_PUBLIC_URL	nessuno	URL esterno usato nei link email.
MTA_ALLOW_INSECURE_NO_AUTH	false	Solo sviluppo isolato.
MTA_MAX_UPLOAD_MB	150	Limite upload in MB. Il wizard Kubernetes usa lo stesso valore anche per l'Ingress.
MTA_DATA_DIR	/data/projects	Storage persistente.
XDG_CACHE_HOME	/data/projects/.cache	Cache modello/plugin.
TORCH_HOME	/data/projects/.cache/torch	Cache Torch/Demucs.
MTA_PORT	8080	Porta HTTP.
MTA_APP_VERSION	VERSION	Versione applicativa.
MTA_BUILD_ID	BUILD_INFO	YYYYMMDD-HH:MM:SS.

5. Demucs/stem plugin

Il registry plugin espone stato e modelli supportati. L'adapter Demucs esegue il comando con argv fisso, senza shell. I modelli consentiti sono `htdemucs`, `htdemucs_ft` e `htdemucs_6s`. Il modello 6-stems è raccomandato quando sono richiesti guitar e piano separati.

Il primo utilizzo può richiedere accesso a Internet per ottenere i pesi. In cluster air-gapped predisporre la cache sul PVC prima dell'avvio operativo.

6. Pipeline audio e plugin

Gli insert sono mappati esclusivamente a preset FFmpeg presenti in un allow-list server-side. Il client non può inviare espressioni FFmpeg arbitrarie. Ogni traccia può avere fino a 16 insert EQ/normalizer/compressor/limiter. Il master applica una seconda catena separata sugli export stereo e sulle preview.

Export	Pipeline
MTA	Clip → insert traccia → fader/pan → mapping slot/merge → stream separati → container Matroska/MTA. Il progetto non è limitato a 8/16 tracce durante il mix.
WAV/ MP3/ FLAC	Clip → insert traccia → fader/pan → summing → master fader → master inserts → encode. Le singole tracce sono esportabili separatamente in WAV/MP3/FLAC.

Reverse analysis MTA v3: all'import il sistema genera un report Matroska/attachment e decodifica le parti dimostrate dei file reali: frame testuali ID3v2.3, DELTA, wrapper LYRICS/COLORS/MIDITK/CHORDS e XML MtxInfoData. Il decoder recupera una keystore condivisa di 256 byte dalla sezione COLORS. LYRICS e CHORDS ricostruiscono timestamp minuto/centesimi e testo/accordi tramite XOR 0x30. COLORS usa record da 15 byte con quattro cifre tempo, tre cifre di posizione progressiva di highlighting, carry del minuto e record terminale distinto. MIDITK viene ricostruito byte-per-byte come Standard MIDI, salvato come sidecar `.miditk.mid` e ne vengono inventariati formato, PPQ, tempo e Marker meta-event. Il sidecar è esposto da GET `/api/projects/{pid}/mta-miditk`. Il trasporto media proprietario XOR è ora implementato in lettura e scrittura; gli attachment importati vengono preservati byte-per-byte. La generazione di nuovi metadati proprietari segue solo le semantiche documentate e resta distinta dalla qualificazione su hardware reale. Se FFmpeg non riesce a estrarre gli attachment, un fallback EBML FileData usa le dimensioni dichiarate da FFprobe per recuperarli in modo conservativo.

7. Sicurezza

Controllo	Implementazione
Auth	Sessioni HttpOnly multi-utente, ruoli admin/user, conferma email, approvazione amministrativa e TOTP opzionale. HTTP Basic resta solo compatibilità legacy/operativa.
Request integrity	X-MTA-Request: 1 sulle richieste mutative.
Path traversal	ID e filename confinati nelle directory progetto.
Upload	Limite, estensione e FFprobe.
Plugin processing	Preset allow-listed; niente filtri arbitrary dal browser.
Process execution	FFmpeg, FFprobe e Demucs senza shell.
HTTP headers	CSP, nosniff, frame deny, no-referrer, Permissions-Policy.
Container	Non-root, cap-drop ALL, seccomp e no privilege escalation.

7.1 Gestione utenti, TOTP e SMTP

Il database autenticazione è `/data/projects/auth.sqlite3` e viene incluso nel backup del PVC. Al primo avvio l'app crea l'amministratore bootstrap solo se sono definiti `MTA_ADMIN_USERNAME`, `MTA_ADMIN_PASSWORD` e `MTA_ADMIN_EMAIL`.

Flusso	Regola
Registrazione	Email obbligatoria, account inizialmente inattivo.
Conferma email	Token monouso con scadenza 24 ore.
Attivazione	Solo un admin può attivare; il server rifiuta l'attivazione se l'email non è confermata.
Notifica	All'attivazione viene inviata una mail a tutti gli admin attivi con email confermata, se SMTP è configurato.
TOTP	RFC 6238/SHA-1, passo 30 s, 6 cifre; secret e QR generati dall'app.
SMTP	Plain SMTP, STARTTLS o SMTPS; autenticazione username/password opzionale. La password salvata non viene restituita dalle API.

Il pannello `/admin/users` consente gestione account e configurazione SMTP. Impostare `MTA_PUBLIC_URL` quando l'app è pubblicata dietro Ingress/reverse proxy, così i link di conferma e reset email contengono l'URL esterno corretto.

Workspace, condivisione e dump amministrativo

Ogni progetto contiene `owner_user_id` e l'elenco degli utenti con cui è condiviso. I progetti precedenti all'introduzione del multi-utente vengono assegnati al primo amministratore persistente che apre il workspace dopo l'upgrade.

Il pulsante **Dump completo progetti** nella pagina amministrativa genera un archivio ZIP organizzato per utente. Il dump include ogni directory di progetto completa, quindi anche `project.json`, audio, attachment, `source.mta`, `directory originals/` e altri artefatti del progetto. Il database credenziali e le password SMTP non vengono inclusi nel dump dei progetti.

Il limite upload Kubernetes è 150 MB per default. Il wizard espone l'opzione `--max-upload-mb` e configura coerentemente `MTA_MAX_UPLOAD_MB` e le annotazioni NGINX/HAProxy Ingress.

8. CI/CD

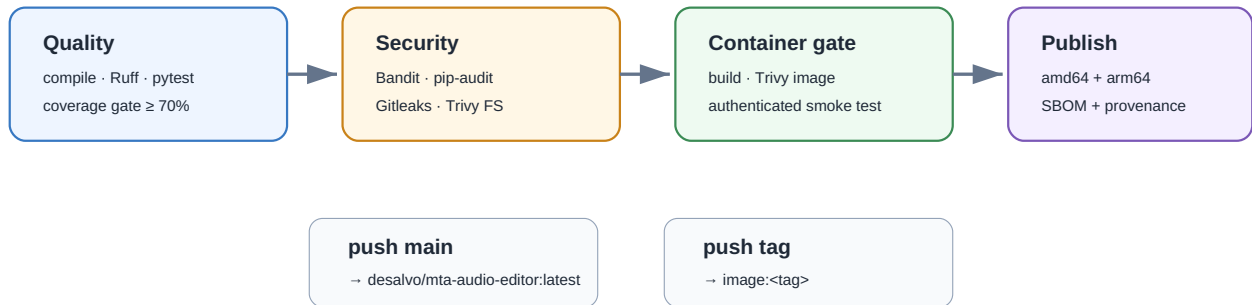


Figura 2 - Gate obbligatori prima della pubblicazione.

Quality, test/coverage, Ruff, Bandit, pip-audit, Gitleaks, Trivy filesystem/image, container smoke test e CodeQL. CI e immagine production usano Python 3.14. Un push su main pubblica `desalvo/mta-audio-editor:latest`; un tag pubblica lo stesso tag. Le immagini sono amd64/arm64 con SBOM e provenance.

9. Operazioni

- Terminare TLS su Ingress/reverse proxy.
- Monitorare spazio PVC: sorgenti audio, export e cache modello possono occupare molto spazio.
- Dimensionare CPU/RAM in base alla separazione stems; Demucs è sensibilmente più pesante del normale editing.
- Centralizzare i log e monitorare probe/restart.
- Mantenere aggiornati base image e dipendenze tramite Dependabot e gate.

10. Backup/restore

Eseguire snapshot/backup coerenti del PVC. Il dato essenziale è `/data/projects`. La sottodirectory `.cache` può essere esclusa per ridurre il backup, sapendo che i modelli dovranno essere riscaricati o ripristinati separatamente.

11. Release e versioning

Release corrente: **0.2.0-60**. Ogni package aggiorna automaticamente BUILD_INFO nel formato YYYYMMDD-HH:MM:SS. Esempio:

```
./scripts/package.sh
git add .
git commit -m "Release 0.2.0-60"
git push origin main
git tag -s 0.2.0-60 -m "MTA Audio Editor 0.2.0-60"
git push origin 0.2.0-60
```

12. Responsabilità production

La baseline include hardening, test e gate automatizzati. Restano responsabilità dell'operatore: TLS, segreti, backup, monitoraggio, network policy, capacity planning, patch management e validazione funzionale sui dispositivi MTA reali. La struttura on-disk verificata è documentata e il trasporto media è implementato; la certificazione su tutte le revisioni firmware Merish resta una validazione esterna su hardware reale.

12.1 Reverse engineering MTA controllato

Artefatto	Analisi	Politica di scrittura
Container	FFprobe/Matroska: stream, codec, tag, attachment.	Ricostruzione controllata in export.
SYL / attachment proprietario	ID3v2.3 completo, frame testuali, DELTA e fingerprint dei payload.	Preservato byte-per-byte finché il campo non è documentato o validato.
MtxInfoData XML	Identity, General, tracce, routing/volume/mute e statistiche NoteOn; padding terminale rilevato.	Lettura strutturata; nessuna inferenza nascosta sulla soglia NoteOn.
LYRICS / CHORDS / COLORS / MIDITK	Decodifica read-only validata su quattro file stock: LYRICS/CHORDS con testo/accordi e timing minuto/centesimi; COLORS con record da 15 byte, timing e posizione progressiva; MIDITK ricostruito come Standard MIDI con tempo/marker.	Il sentinel COLORS 127 è trattato come reset/cambio pagina; NoteOn come maschera di attività sorgente a un secondo. Gli attachment importati sono comunque preservati per round-trip conservativo.

Per chiudere la compatibilità di scrittura creare coppie di MTA dello stesso brano variando un solo elemento alla volta (una parola, un accordo, un marker, un colore). L'endpoint `/api/mta/diff` restituisce prefisso/suffisso comune e range binari modificati.

13. Plugin custom e Auto Mix

13.1 Registry e preset

I processori sono definiti nel registry server-side `app/plugins.py`. Il browser non invia espressioni FFmpeg arbitrarie: invia solo tipo plugin, preset o parametri numerici validati rispetto a uno schema con range min/max. I preset custom sono persistiti in `/data/projects/_custom_presets.json`; includere questo file nei backup del PVC.

Controllo	Implementazione
Factory preset	Immutabili, definiti nel codice.
User preset	Parametri JSON validati e salvati sul volume persistente.
Raw DSP expression	Non accettata dalle API.
Catena	Massimo 16 insert per traccia/master, applicati in ordine.
Lexicon-style	Preset ispirati a plate/hall/ambient usando filtri FFmpeg aperti; nessun algoritmo proprietario Lexicon.

13.2 Auto Mix

L'endpoint `POST /api/projects/{pid}/auto-mix` riceve `{enabled, style}`. All'attivazione salva uno snapshot completo di volume, pan e insert di ogni traccia più volume/insert master. Le regole sono deterministiche e type-aware; alla disattivazione lo snapshot viene ripristinato. Questo evita drift cumulativo e rende il wizard idempotente dal punto di vista operativo.

13.3 Preset API

```
POST /api/presets
{
  "plugin": "amplify",
  "name": "My Trim",
  "params": {"gain_db": -3.0}
}

DELETE /api/presets/amplify/My%20Trim
```

Job di import/export traccia e stima BPM

Gli import di traccia usano job asincroni consultabili tramite `/api/media-jobs/{job_id}`. L'upload espone avanzamento dal browser; il backend prosegue con validazione, eventuale stima BPM, sincronizzazione e commit del progetto. Solo il primo MP3 di un progetto viene analizzato per il BPM. Gli import successivi non cambiano il valore.

Gli export di singola traccia usano analogamente un job di rendering con progress e un endpoint di download associato. Il percorso del file risultante è vincolato alla directory del progetto.

Mute/Solo sono proprietà di mix, non filtri di visualizzazione: tutte le tracce rimangono presenti nel modello, nella timeline e nel mixer.

Workflow persistente Import & Separate

La separazione stem è implementata come job asincrono cancellabile. L'upload MP3 viene prima validato, copiato nel workspace e negli originali, aggiunto come traccia **Original Mix** e il progetto viene salvato. Solo dopo parte Demucs.

```
POST /api/stems/jobs
GET /api/stems/jobs/{job_id}
POST /api/stems/jobs/{job_id}/cancel
```

Il worker aggiorna percentuale/fase, aggiunge gli stem al progetto e salva dopo ogni traccia. Un cancel usa un `threading.Event`, termina/uccide il subprocess Demucs se necessario e fa rollback degli stem parziali creati dal job. Per evitare race sul medesimo `project.json` è ammesso un solo job attivo per progetto.

Lo stato del job è volatile, mentre progetto, MP3 originale e file già consolidati sono sul PVC. Un restart del pod interrompe il job attivo ma non cancella il progetto.

Persistenza e autosave dei progetti

La creazione di un progetto scrive immediatamente `project.json`. Le operazioni backend salvano al termine della mutazione; i controlli puramente UI usano autosave debounced. Prima di cambiare progetto o di render/export viene effettuato un flush dell'autosave pendente.

L'utente può inoltre scaricare un archivio locale completo del progetto oppure cancellarlo dal workspace. La cancellazione è owner-only (oltre agli amministratori); un collaboratore condiviso non può eliminare il progetto.

Waveform persistenti e motore di preview

Le waveform sono cache persistenti nel modello Track (`waveform_peaks` e `waveform_revision`). Un job asincrono rigenera i picchi quando la revision del file audio non coincide, salva il progetto e pubblica avanzamento tramite la stessa infrastruttura / `api/media-jobs`.

Il layout editor salva `track_panel_width_px` nel progetto. Tracks e timeline mantengono scroll verticali sincronizzati; lo stato UI temporaneo conserva inoltre scroll della timeline e del mixer durante i rerender necessari.

RealTime usa Web Audio API/AnalyserNode per misurare RMS istantaneo delle preview delle singole tracce. Il meter dinamico è separato dall'indicatore statico di fader.

La preview Render usa il master render server-side per includere insert di traccia, fader/pan, mute/solo, master chain e trasformazioni globali. Quando RealTime è attivo vengono riprodotte in parallelo preview di traccia silenziate in uscita, usate soltanto come sorgenti dei meter.

Tempo e pitch globali

`base_bpm` conserva il BPM di riferimento; `bpm` è il target corrente. Il rapporto determina il time-stretch. `pitch_semitones` è limitato a -6/+6. La trasformazione FFmpeg usa `asetrate + aresample` e una catena atempo per rendere pitch e tempo indipendenti. La trasformazione è applicata a preview, mix export, single-track export e slot MTA.

Cancellazione progetto

Il backend elimina l'intera directory del progetto. La UI richiede due conferme consecutive, inclusa una conferma esplicita di irreversibilità. Non esiste cestino applicativo: il recupero richiede un backup/snapshot esterno.

Build native Windows x64/macOS ARM64

La pipeline CI/CD crea anche applicazioni desktop native. Il launcher usa PyInstaller + pywebview e avvia FastAPI/Uvicorn soltanto sull'interfaccia loopback `127.0.0.1`.

Le versioni native sono mono-utente e disabilitano login, registrazione, gestione account/utenti, SMTP/TOTP e sharing. I dati persistono in %LOCALAPPDATA%\MTA Audio Editor su Windows e ~/Library/Application Support/MTA Audio Editor su macOS.

La pipeline installa e incorpora FFmpeg/FFprobe e Demucs/PyTorch, esegue uno smoke test dell'eseguibile frozen e poi produce un installer Inno Setup su Windows e un DMG su macOS. Su tag, entrambi vengono pubblicati automaticamente nella GitHub Release.

Native Demucs, iconografia e metadata

Nelle build frozen Windows/macOS Demucs viene invocato in-process dal worker thread backend. Non viene più rilanciato l'eseguibile GUI con `--demucs-worker`: questo evita su macOS la creazione/terminazione di una seconda istanza applicativa durante il caricamento del modello.

Le build PyInstaller includono l'icona applicativa in formato ICO/ICNS. Windows riceve anche version resources generate da `native/generate_native_metadata.py`; macOS inserisce nel bundle plist versione, build, creatore e repository.

L'endpoint `/api/about` resta la fonte applicativa per nome, versione, build, creatore, repository e licenza; la stessa informazione è esposta dal dialog About nelle varianti web e native.

Stem job da traccia esistente

L'endpoint `POST /api/projects/{pid}/tracks/{track_id}/stem-jobs` avvia la separazione Demucs direttamente sul working audio della traccia selezionata. Non viene creato un nuovo upload e il file sorgente non viene eliminato. Il job riusa lo stesso controller asincrono, progress reporting e cancellazione dei normali stem job.

Follow, layout Mixer e metering L/R

Le preferenze `follow_playback_enabled`, `export_panel_visible` e `metadata_panel_visible` sono persistite nel modello Project. Il layout del Mixer ridistribuisce le colonne in base ai pannelli visibili.

Il metering RealTime usa Web Audio API. Le sorgenti mono usano un singolo `AnalyserNode`; le sorgenti stereo vengono separate tramite `ChannelSplitterNode(2)` e misurate con `analyser` indipendenti L/R. Il Master è sempre stereo. In Render mode il master renderizzato alimenta direttamente gli `analyser` Master L/R; in dynamic preview il valore Master viene composto dai livelli L/R delle tracce attive.

Il catalogo pubblico non espone più l'insert legacy eq. Il supporto di rendering legacy resta disponibile per aprire progetti preesistenti. Auto Mix usa `graphic_eq_32`. L'editor del Graphic EQ è una superficie grafica a 32 fader, mentre la validazione server-side continua ad applicare i limiti -12/+12 dB per banda.

Native save binding e multiprocessing

Il bridge pywebview espone `choose_project_save_path`, `bind_project_path` e `sync_project`. Il frontend richiede il percorso prima di creare un nuovo progetto tramite Import & Separate, quindi associa il percorso al project ID e sincronizza l'archivio esterno a ogni salvataggio e alla fine del job.

Il launcher frozen invoca `multiprocessing.freeze_support()` prima di `main()`. Questo è necessario perché Demucs/PyTorch possono creare processi worker: senza il dispatch PyInstaller, il child può rieseguire il launcher e creare una seconda finestra GUI vuota.

Appendice A. Specifica tecnica del formato proprietario MTA

Ambito: questa appendice documenta il formato on-disk verificato sul corpus stock disponibile. "Verificato" significa che la relazione è stata ricostruita dai byte e controllata su più file indipendenti. La compatibilità con ogni revisione firmware M-Live/Merish richiede comunque una prova sul dispositivo target.

A.1 Organizzazione generale

MTA è un contenitore derivato da Matroska. Prima del primo Cluster le strutture EBML sono leggibili normalmente. L'ordine stock osservato è:

```
EBML Header
Segment
  SeekHead
  Void
  Info
  Tracks
  Attachments
  Cues
  Cluster <-- da qui inizia il trasporto XOR
  Cluster
  ... fino a EOF
```

SeekPosition e CueClusterPosition sono relativi all'inizio dei dati del Segment.

A.2 Trasporto media XOR

Dal primo byte del primo Cluster fino a EOF viene applicato uno XOR periodico fisso di 984 byte:

```
plain[i] = stored[i] XOR key[i mod 984]
stored[i] = plain[i] XOR key[i mod 984]
```

La fase parte da zero sul primo byte 1F 43 B6 75 del Cluster e non viene mai riavviata. SHA-256 della chiave:

```
bcb30443707bdc8b651c1a58aa4152438ce5a6632cc98b294501eb08adbb3547
```

La costante completa è mantenuta nel codice come MEDIA_XOR_KEY. Lo XOR è simmetrico: la stessa routine converte MTA memorizzato in Matroska canonico e viceversa.

A.3 EBML essenziale

Elemento	ID EBML	Uso
Segment	18538067	Contenitore principale.
SeekHead	114D9B74	Indice delle strutture principali e del primo Cluster.
Info	1549A966	Timebase e informazioni generali Matroska.
Tracks	1654AE6B	Descrizione tracce audio.
Attachments	1941A469	SYL e MtxInfoData.
Cues	1C53BB6B	Indice temporale dei Cluster.
Cluster	1F43B675	Pacchetti media.
SimpleBlock	A3	Un frame MP3 per blocco.

A.4 Audio MP3 e padding

Le tracce stock verificate sono MPEG-1 Layer III, 44.1 kHz, stereo, 320 kbit/s. Il primo pacchetto di ogni traccia è un frame Info non padded da 1044 byte. I frame musicali successivi sono 1044 o 1045 byte.

```
padding[0] = 0
state = 4
per ogni frame musicale:
    state += 44
    if state >= 49:
        padding = 1
        state -= 49
    else:
        padding = 0
```

A.5 SimpleBlock e Cluster

Ogni SimpleBlock contiene un solo frame MP3 ed ha normalmente 7 byte di overhead:

```
A3 | size-VINT | track-VINT | relative-timecode-int16 | 80 | MP3 fram
```

L'ordine è **frame-major**: a ciascun istante vengono scritte tutte le tracce, poi si passa all'istante successivo. Un Cluster completo contiene 10 frame-time per traccia. Timecode relativi:

```
0, 144, 288, 432, 576, 720, 864, 1008, 1152, 1296
```

Parametro	Valore verificato
TimecodeScale	181392 ns
DefaultDuration	26122448 ns
Cluster timecode	$\text{floor}(i * (8000000000 / 555513) + 1/16)$

L'ultimo Cluster può essere parziale e usa sempre la larghezza VINT EBML minima necessaria.

A.6 Cues e SeekHead

Viene emesso un Cue ogni quattro Cluster, cioè ogni 40 pacchetti MP3 per traccia:

```
cue_count = ceil(packet_count_per_track / 40)
Cue n -> Cluster 4*n
```

Le dimensioni di Seek/Cue dipendono dalla larghezza minima degli offset; un writer rigoroso risolve quindi il layout con iterazione a punto fisso. MTA Audio Editor normalizza il Matroska generato ponendo SeekHead e Cues prima del primo Cluster, quindi applica il trasporto XOR soltanto alla regione media.

A.7 Frame MP3 Xing/Info

Offset	Campo
36..39	Info
40..43	flags 0000000F
44..47	numero frame musicali, Info escluso
48..51	byte MP3 totali, Info incluso
52..151	TOC Xing, 100 byte
152..155	quality
156..	encoder string
177..179	encoder delay/padding, 12+12 bit
184..187	music length
188..189	music CRC
190..191	tag CRC

`Info.frames = pacchetti_totali - 1`; `Info.bytes = music_length` = somma dei byte di tutti i frame MP3 incluso Info.

La TOC usa l'algoritmo FFmpeg/LAME con bag intermedio da 400 campioni. Entrambi i CRC sono CRC-16/ANSI riflessi con polinomio 0xA001 e inizializzazione zero: il music CRC copre i frame musicali; il tag CRC copre i byte 0..189 del frame Info dopo l'inserimento del music CRC.

A.8 Attachment stock

I file verificati contengono quattro attachment: SYL originale, SYL modificato, MtxInfoData XML originale e XML modificato. Nel corpus analizzato le coppie original/modificato sono byte-identiche. In import l'applicazione conserva gli attachment originari e non li perde durante la gestione del progetto.

A.9 SYL e keystore proprietario

SYL è un ID3v2.3 completo seguito dalle sezioni M-Live DELTA, LYRICS, COLORS, MIDITK, CHORDS. Le sezioni proprietarie condividono una keystore XOR di 256 byte, identica sui file stock verificati e costituita da una permutazione completa di 00..FF.

Prefisso binario comune delle sezioni osservate: 28 B6 A9.

A.10 COLORS

I record normali sono lunghi 15 byte. Plaintext:

```
LF d d RS d d 'm' p p p '=' ':' 'k' NUL next_minute
```

Timing e posizione:

```
centiseconds = d1*1000 + d2*100 + d3*10 + d4
time_ms      = minute_block*60000 + centiseconds*10
position     = p1*100 + p2*10 + p3

key_index = (record_index - 17*byte_index) mod 256
stored    = plain XOR key[key_index] XOR 0x0A
```

Il valore posizione **127** è un controllo di reset/cambio pagina karaoke, non un indice carattere. Compare alla transizione fra la riga/pagina precedente e la prima riga della pagina successiva.

A.11 LYRICS e CHORDS

Usano una fase lineare della stessa keystore:

```
key_index = 239 * flat_payload_offset mod 256
stored    = plain XOR key[key_index] XOR 0x0A
```

I byte di testo/accordo applicano inoltre XOR 0x30. I record terminano nel marker decodificato = :k\0X, dove X porta il blocco minuto successivo. Il timing è minuto + centesimi come in COLORS.

A.12 MIDITK

La decodifica è:

```
key_index = 239 * (payload_offset - 3) mod 256
midi_byte = stored XOR key[key_index] XOR 0x0A XOR 0x30
```

Il risultato è un Standard MIDI File byte-valido: formato 0, una traccia, PPQ 480, eventi tempo e marker musicali, più End Of Track.

A.13 MtxInfoData XML e NoteOn

Il documento XML contiene Identity, General e TrkAudio/Track. Sono stati verificati BaseKey, BaseBpm, DurataSec, PreCntUSec, Num, Nome, Type, Routing, Volume, MuteSolo e NoteOn.

NoteOn è una maschera di attività sorgente/pre-encode a granularità circa un secondo. Sul corpus: 4 brani, 50 tracce, 12.654 bin. Una separazione RMS ottimale sul post-MP3 ricostruisce oltre il 99,8% dei bit; gli scarti si concentrano ai bordi di attività e sul materiale molto debole. Per i nuovi progetti l'app deve quindi generare NoteOn dalla timeline/sorgente prima dell'encoding, non tentando di imitare una soglia MP3 universale inesistente.

PreCntUSec è una proprietà esplicita di pre-count/lead-in in microsecondi; non è riducibile a una formula basata soltanto sul BPM arrotondato.

A.14 Algoritmo di import

1. preserva il file MTA originale
2. parse EBML pre-media
3. trova il primo Cluster
4. verifica se il Cluster raw è canonico
5. se non canonico, prova XOR-984 sul prefisso
6. se validato, crea una vista Matroska temporanea deobfuscata
7. FFprobe/FFmpeg estraggono le tracce dalla vista canonica
8. attachment originali vengono preservati
9. SYL/XML vengono analizzati separatamente
10. project.json viene creato senza distruggere gli originali

A.15 Algoritmo di export implementato

1. render non distruttivo delle tracce
2. mapping/merge negli slot MTA8/MTA16
3. creazione Matroska canonica
4. normalizzazione top-level: SeekHead + Void + Info + Tracks + Tags/
5. rigenerazione CueClusterPosition dopo lo spostamento dei Cues
6. aggiunta SeekHead verso il primo Cluster
7. append dei Cluster canonici
8. XOR-984 dal primo Cluster a EOF
9. validazione del primo Cluster decodificato
10. test CI di round-trip mediante deobfuscazione + FFprobe

A.16 Livelli di compatibilità

Livello	Stato
Lettura strutturale	Implementata e testata.
Lettura proprietaria media	XOR-984 implementato e testato.
Lettura proprietaria SYL/XML	Implementata sugli invarianti verificati.
Scrittura trasporto proprietario	Implementata con normalizzazione Cues/SeekHead e XOR.
Round-trip software	Validato: MTA scritto → deobfuscazione → FFprobe/import.
Preservazione metadata importati	Implementata.
Qualificazione hardware universale	Richiede test su specifico modello/firmware M-Live/Merish.

Appendice B. Progetto software, dati e flussi

B.1 Moduli principali

Modulo	Responsabilità
main.py	FastAPI, pagine, API, middleware, autorizzazione.
auth.py	Utenti, sessioni, verifica email, TOTP, SMTP.
models.py	Modello Pydantic di progetto, tracce, clip, insert, eventi.
storage.py	Workspace, path confinement, condivisioni, archivi completi.
audio_engine.py	Render clip, filtri, mix, sync, preview/export.
plugins.py	Registry DSP allow-listed e preset.
auto_mix.py	Auto Mix deterministico e reversibile.
codec.py	Import/export MTA operativo.
mta_writer.py	Normalizzazione Matroska per layout MTA con Cues pre-media.
mta_reverse.py	Analisi byte-level, XOR, SYL/XML, MIDITK.

B.2 Persistenza

```
/data/projects/  
  auth.sqlite3  
  .auth-secret  
  _custom_presets.json  
  .cache/  
    <project-id>/  
      project.json  
      audio/  
      attachments/  
      originals/  
      source.mta  
      mta-analysis.json
```

audio/ contiene le sorgenti di lavoro, mentre originals/ conserva gli upload originali. I dump completi di progetto includono gli originali.

B.3 Modello di editing

Le modifiche sono non distruttive. Ogni Clip indica intervallo della sorgente e posizione timeline. Tagli e ripple modificano i riferimenti, non i file originali. La pipeline di rendering è:

```
clip -> insert traccia -> fader/pan -> merge slot MTA  
clip -> insert traccia -> fader/pan -> somma stereo -> master -> expo
```

B.4 Sicurezza applicativa

La superficie web usa sessioni HttpOnly, SameSite=Lax, Secure quando HTTPS è rilevato, ruoli admin/user, verifica email, approvazione admin e TOTP opzionale. Le mutazioni API richiedono anche X-MTA-Request: 1. Il browser non può inviare filtri FFmpeg arbitrari: plugin e parametri sono validati contro registry/schema server-side.

B.5 Workspace e condivisioni

Ogni progetto ha owner_user_id e shared_with_user_ids. Gli utenti vedono soltanto i progetti propri o condivisi. L'import di un archivio completo crea un nuovo ID, assegna il progetto all'importatore e azzerà le condivisioni pregresse.

B.6 Kubernetes

Il wizard genera Namespace, PVC, Secret, Deployment, Service e Ingress opzionale. Supporta limite upload, StorageClass, nodeSelector, TLS, NGINX/HAProxy e imagePullPolicy. Il pod usa UID/GID 10001 e fsGroup: 10001; startup/readiness/liveness interrogano /api/health.

B.7 CI/CD e criteri di release

I gate coprono compilazione, pytest/coverage, Ruff, Bandit, audit dipendenze, secret scanning, CodeQL, Trivy filesystem/immagine e smoke test del container. Il test MTA di writer deve verificare che un file generato esponga il trasporto XOR, che la deobfuscazione sia FFprobe-readable e che l'import ricostruisca il numero corretto di tracce.

La documentazione progettuale estesa è inoltre disponibile nei file docs/PROJECT_ARCHITECTURE.md, DATA_MODEL.md, API_REFERENCE.md, OPERATIONS_RUNBOOK.md, TESTING_AND_RELEASE.md e MTA_FORMAT_FINAL_SPEC.md.

Cancellazione progetto: La cancellazione di un progetto elimina l'intero workspace del progetto, inclusi audio, originali, attachment e file generati; l'interfaccia richiede sempre due conferme consecutive, la seconda esplicitamente irreversibile.

Client mobile e distribuzione

I client in `mobile/android` e `mobile/ios` sono thin native clients del backend. Per produzione usare HTTPS e un certificato attendibile. La CI genera APK/AAB e xcarchive/IPA; i secret richiesti per firma e TestFlight sono documentati in `docs/MOBILE_APPS.md`.

I download mobile riutilizzano i cookie autenticati del WebView/WKWebView e i job `configured-export-jobs`, evitando di esporre file senza autorizzazione.